

PLP - Primer Parcial - 2^{do} cuatrimestre de 2025

#Orden	Turno	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
				E	E	MB	(P)

Este examen se aprueba obteniendo al menos dos ejercicios bien (B) y uno regular (R), y se promociona con al menos dos ejercicios muy bien (MB) y uno bien (B). Es posible obtener una aprobación condicional con un ejercicio muy bien (MB), uno bien (B) y uno insuficiente (I), pero habiendo entregado algo que contribuya a la solución del ejercicio. Las notas para cada ejercicio son: -, I, R, B, MB, E. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Entregar cada ejercicio en hojas separadas. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación funcional

Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.

El siguiente tipo de datos sirve para representar un sistema de archivos sin enlaces lógicos (lo llamamos FS por su sigla en inglés). Un sistema de archivos puede entenderse como un árbol de subdirectorios y archivos con la siguiente estructura:

```
data FS = Arch String | Dir String [FS] deriving (Eq, Show)
```

Donde Arch nombre representa el archivo con nombre nombre, y Dir nombre contenido representa el directorio llamado nombre que contiene los archivos y subdirectorios que aparecen en la lista contenido.

Definimos el siguiente directorio para los ejemplos:

```
unDir = Dir "raíz" [Arch "arch1", Arch "arch2", Arch "arch3",
  Dir "subdir1" [Arch "arch4"], Dir "subdir2" [Arch "arch1", Arch "arch5"]]
```

a) Dar el tipo y definir las funciones foldFS y recFS, que implementan respectivamente los esquemas de recursión estructural y primitiva para el tipo FS. Solo en este inciso se permite usar recursión explícita. Ayuda: `recFS :: ... -> (String -> [FS] -> [a] -> a) -> ...`

b) Definir la función rutas :: FS -> [String], que lista las rutas de todos los archivos y subdirectorios de un sistema de archivos dado, usando la barra "/" como separador de directorios.

Por ejemplo:

```
rutas unDir ~ ["raíz", "raíz/arch1", "raíz/arch2", "raíz/arch3", "raíz/subdir1",
  "raíz/subdir1/arch4", "raíz/subdir2", "raíz/subdir2/arch1", "raíz/subdir2/arch5"]
```

c) Definir la función válido :: FS -> Bool, que indique si, para cada directorio de un sistema de archivos, todos sus archivos y subdirectorios tienen nombres distintos. No importa si hay nombres repetidos entre distintos niveles - por ejemplo, no hay problema si un directorio tiene el mismo nombre que uno de sus archivos o subdirectorios - o si dos subdirectorios contienen archivos con el mismo nombre; solo importan los nombres dentro de un mismo directorio.

Por ejemplo:

```
válido unDir ~ True
```

```
válido (Dir "dir1" [Arch "arch1", Arch "arch1"]) ~ False
```

```
válido (Dir "dir1" [Dir "dir2" [Arch "arch1", Dir "arch1" []]]) ~ False
```

Sugerencia: usar all y nub.

d) Definir la función rutasPosibles :: [String] -> [String], que dada una lista de nombres (sin repetidos) genera la lista (infinita) de todas las rutas no vacías que se pueden generar con esos nombres.

Por ejemplo:

```
rutasPosibles ["A", "B", "C"] debe contener a "A", "B", "C", "A/A", "C/B",
  "A/C/C", "B/A/A", "B/A/B", "C/A/B", "A/A/B/B", "C/B/C/A" entre otras.
```

El código debe tener la siguiente estructura (a completar):

```
rutasPosibles xss = concatMap ... [0..]
  where
    ... = foldNat (\rs -> [s1++"/":s2 | ...]) ...
```

Con foldNat :: (b -> b) -> b -> Integer -> b definido en la guía.

Ejercicio 2 - Demostración de propiedades

Considerar las siguientes definiciones¹:

```
data Árbol23 a b = Hoja a | Dos b (Árbol23 a b) (Árbol23 a b)
                  | Tres b b (Árbol23 a b) (Árbol23 a b) (Árbol23 a b)

const :: a -> b -> a
{C} const = (\x -> \y -> x)
tamaño :: Árbol23 a b -> Int
{A0} tamaño (Hoja v) = 1
{A1} tamaño (Dos x i d) = 1 + tamaño i + tamaño d
{A2} tamaño (Tres x y i m d) = 2 + tamaño i + tamaño m + tamaño d
truncar :: a -> Árbol23 a b -> Int -> Árbol23 a b
{TO} truncar c (Hoja v) = const (Hoja c)
{TI} truncar c (Dos x i d) =
    (\n->if n<1 then Hoja c else Dos x (truncar c i (n-1)) (truncar c d (n-1)))
{T2} truncar c (Tres x y i m d) =
    (\n->if n<1 then Hoja c else Tres x y (truncar c i (n-1)) (truncar c m (n-1)) (truncar c d (n-1)))
```

a) Demostrar la siguiente propiedad:

$$\forall t :: \text{Árbol23 } a \ b. \forall c :: a. \forall n :: \text{Int}. \text{tamaño} (\text{truncar } c \ t \ n) \leq \text{tamaño } t$$

Es importante plantear la propiedad como predicado unario y exhibir claramente el esquema de inducción estructural.

Se consideran demostradas las propiedades conocidas sobre enteros y booleanos, así como también:

$$\{\text{LEMA}\} \forall t :: \text{Árbol23 } a \ b. 1 \leq \text{tamaño } t$$

Para el caso del constructor Tres no es necesario escribir la demostración paso por paso, basta con enunciar la hipótesis inductiva y la tesis inductiva. Para los constructores Hoja y Dos sí es importante presentar la demostración completa.

b) Demostrar el siguiente teorema usando deducción natural, sin utilizar principios clásicos:

$$((\pi \Rightarrow \rho) \vee (\sigma \Rightarrow \tau)) \Rightarrow (\sigma \wedge \pi) \Rightarrow (\rho \vee \tau)$$

Ejercicio 3 - Cálculo Lambda Tipado

Se desea extender el cálculo lambda para modelar estructuras con forma de **anillos** finitos. Un anillo cuenta con un elemento actual, seguido por una cadena de 0 o más elementos hasta volver al actual.

Se extienden los tipos y expresiones de la siguiente manera:

$\tau ::= \dots \mid \text{Anillo}_\tau \quad M ::= \dots \mid \langle M \rangle \mid M :: M \mid \text{actual}(M) \mid \text{avanzar}(M)$ donde:

- Anillo_τ es el tipo de los anillos con elementos de tipo τ .
- $\langle M \rangle$ es un anillo cuyo único elemento (y por ende su elemento actual) es M .
- $M_1 :: M_2$ es el anillo resultante de agregar el elemento M_2 a continuación del elemento actual de M_1 . Notar que el elemento actual sigue siendo el mismo, y que este operador es un constructor (no realiza ninguna acción sobre el anillo ni sus elementos). Por ejemplo: $(\langle 0 \rangle :: 2) :: 1$ es el anillo cuyo elemento actual es 0 , seguido por 1 , luego 2 para luego volver al 0 .
- $\text{actual}(M)$ es el elemento actual del anillo M .
- $\text{avanzar}(M)$ es el anillo que resulta de avanzar una posición en el anillo M , de manera que su elemento actual es el último que se agregó en M (o el actual de M si tiene un único elemento).

Por ejemplo: $\text{avanzar}(\text{avanzar}((\langle 0 \rangle :: 2) :: 1)) \rightarrow ((2) :: 1) :: 0$

a) Introducir las reglas de tipado para la extensión propuesta.

b) Definir el conjunto de valores y las nuevas reglas de reducción en un paso. Tener en cuenta que no es lo mismo avanzar un anillo con uno, dos o más elementos.

Pista: puede ser necesario mirar más de un nivel de un término para saber a qué reduce.

c) Mostrar paso por paso cómo reduce la expresión: $\text{actual}(\text{avanzar}((\langle 0 \rangle :: \text{Pred}(3)) :: 1))$

d) Definir como macro la función siguiente_τ , que dado un anillo con elementos de tipo τ devuelve el elemento siguiente al actual.

¹Escritas con recursión explícita para facilitar las demostraciones.